

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Name: Rhaiven Jules Cano	Date: September 24, 2026
Section: BSIT 2A	Instructor:

I. Objective

This program is built to help a support team keep track of incident tickets that come from automation bots. It lets the user add a new ticket, view the full list of active tickets, look up one ticket using its Incident ID, and take out a ticket once it has been resolved. It also shows the total number of tickets that are still active, so the user does not have to count them by hand. The whole program runs through a simple text menu, so a user does not need to touch the code to operate it.

II. Program Operations

Add Incident:

```
tickets.append(ticket)
```

The program asks the user to type in the Incident ID, the Bot name, and a short description. These three answers are stored together as one ticket (a dictionary). Before it gets added, the program checks the list to make sure that Incident ID is not already used. Once it is confirmed unique, `tickets.append(ticket)` puts the new ticket at the end of the list.

Display Incidents:

```
for ticket in tickets:  
    print(ticket['incident_id'], ticket['bot'], ticket['description'])
```

This part uses a for loop to go through the tickets list one item at a time. Each time the loop runs, it prints out the Incident ID, Bot name, and description of that ticket so they line up like a small table. If the list has nothing in it, the program prints a message saying there are no active tickets instead of showing a blank table.

Search Incident:

```
if ticket["incident_id"] == incident_id:
```

The user is asked to type the Incident ID they are looking for. The program then checks each ticket in the list one by one and compares its `incident_id` to the one the user typed. As soon as it finds a match, it prints out that ticket's full details and stops looking. If it goes through the whole list without a match, it tells the user the ticket was not found.

Remove Resolved Incident:

```
tickets.remove(ticket)
```

This works almost the same way as the search. The program looks through the list for the ticket with the Incident ID that was typed in. Once that ticket is found, `tickets.remove(ticket)` takes it out of the list for good. This is meant to be used once a ticket has already been solved by the support team and no longer needs to show up as active.

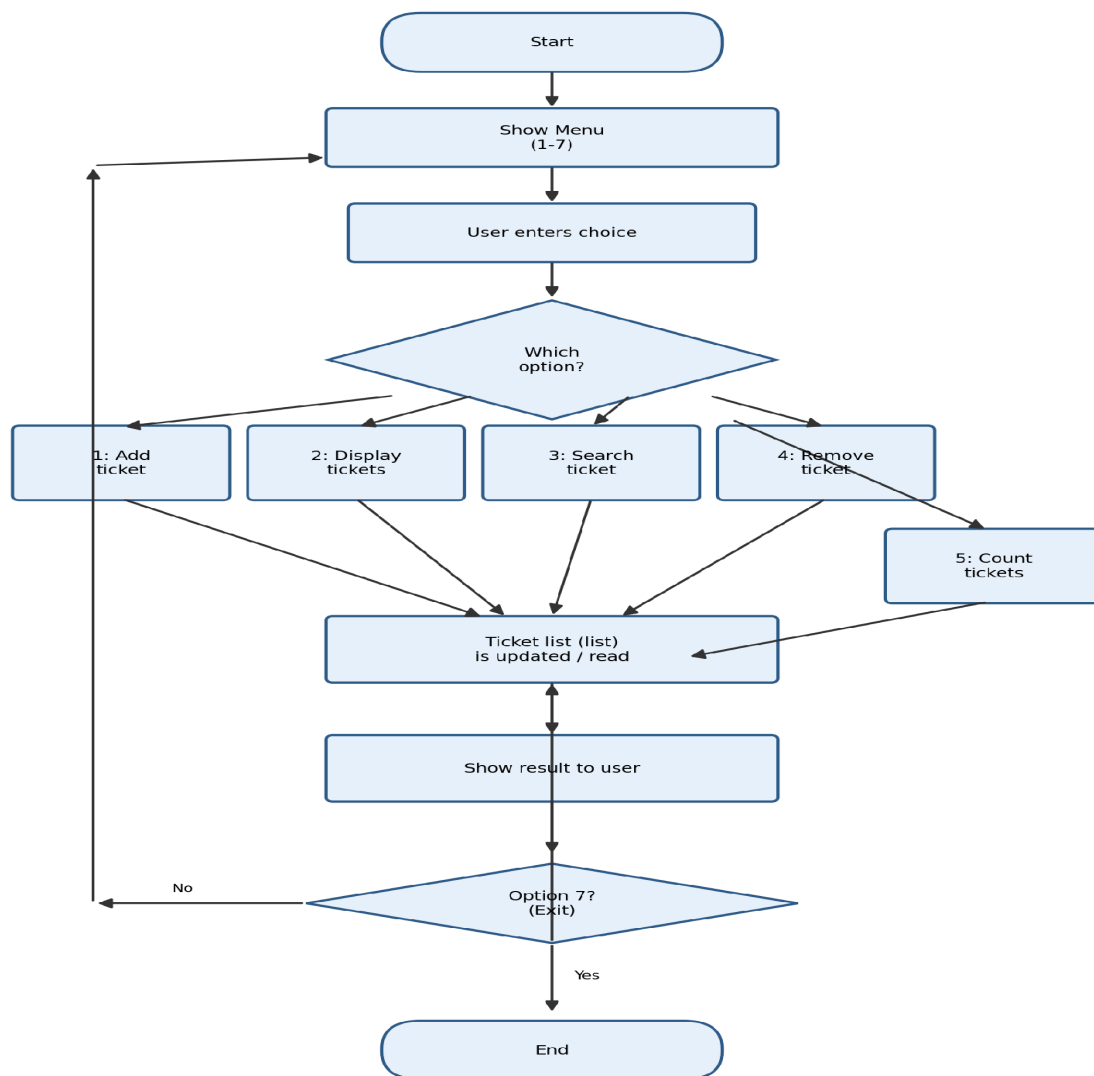
Count Active Incidents:

```
len(tickets)
```

Python's built-in `len()` function is used to count how many tickets are currently sitting inside the tickets list. The program prints this number so the user can see at a glance how many incidents are still open, without needing to scroll through the whole list.

III. Flowchart

The flowchart below shows the general flow of the program: it starts, shows the menu, waits for the user's choice, runs the matching operation on the ticket list, shows the result, and then loops back to the menu unless the user chooses to exit.



IV. Conclusion

Doing this activity helped me see how a linear data structure like a list is not just a topic to memorize but something that actually solves a real problem. Using a list made it simple to add and remove tickets since Python already has built-in tools like `append()`, `remove()`, and `len()` ready to use. I also noticed how much the loop matters when displaying or searching through the tickets, since one small mistake there can make the program show the wrong output. Overall, this exercise made me understand that choosing the right data structure changes how easy or hard the rest of the program becomes to write and to read.